

The amplitude jet remainder

Claude, with Daniel Murfet

2026-09-07

Abstract

For $\xi(u) = \sum_{i < R} x_i u^i + O(u^R)$ and $\eta(u) = \sum_{i < R} y_i u^i + O(u^R)$ on $[0, b]$ the u -jet of the amplitude $\eta(u)e^{\beta s \xi(u)}$ to order R has coefficients $e^{\beta s x_0} P_j(s)$ with $P_j(s) = \sum_{\ell \leq j} y_\ell E_{j-\ell}(s)$, and $|\eta(u)e^{\beta s \xi(u)} - \sum_{j < R} u^j e^{\beta s x_0} P_j(s)| \leq C u^R (1+s)^R e^{\beta s(x_0 + L_1 b)}$. The product with η is handled through polynomial multiplication: the jet sum is the degree $< R$ truncation of $\text{jetPoly}(y) \cdot \text{jetPoly}(E.(s))$, whose coefficients are Cauchy products, and the truncation constant of the product is bounded uniformly in s by a $(1+s)^R$ envelope. This completes Astra #3's target 2b: the compatible finite jets of the 1D amplitude are now available with an explicit remainder of exactly the shape consumed by `oneDKernel_jet_expansion`. Zero sorry/axiom.

1 The things formalised

```
theorem coeff_jetPoly' (x : → ) (R n : ) :
  (jetPoly x R).coeff n = if n < R then x n else 0
theorem natDegree_jetPoly_le (x : → ) (R : ) : (jetPoly x R).natDegree ≤ R - 1
theorem coeff_jetPoly_mul (y E : → ) (R j : ) (hj : j < R) :
  (jetPoly y R * jetPoly E R).coeff j = ∑ range (j + 1), y * E (j - )
theorem abs_coeff_jetPoly_mul_le (y E : → ) (R n : ) :
  |(jetPoly y R * jetPoly E R).coeff n|
    ≤ ((n : ) + 1) * (∑ i range R, |y i|) * 1 range R, |E 1|)
theorem truncConst_jetPoly_mul_le (y E : → ) (R : ) (hR : 0 < R) (b : ) (hb : 0 ≤ b) :
  truncConst (jetPoly y R * jetPoly E R) R b
    ≤ (n Ico R (2 * R), ((n : ) + 1) * b ^ (n - R))
    * (∑ i range R, |y i|) * 1 range R, |E 1|)
noncomputable def ampJet ( : ) (x y : → ) (R j : ) (s : ) : :=
  ∑ range (j + 1), y * expJet x R (j - ) s
theorem ampJet_continuous ( : ) (x y : → ) (R j : ) : Continuous (ampJet x y R j)
theorem eval_gPoly_eq (x : → ) (R : ) (hR : 0 < R) (u : ) :
  (gPoly x R).eval u = (∑ i range R, x i * u ^ i) - x 0
theorem amp_jet_remainder (b K L : ) (h : 0 < ) (hb : 0 < b) (hK : 0 ≤ K)
  (hL : 0 ≤ L) (x y : → ) (R : ) (hR : 0 < R) ( : → )
  (hT : u Set.Icc (0 : ) b, |u - i range R, x i * u ^ i| ≤ K * u ^ R)
  (hT : u Set.Icc (0 : ) b, |u - i range R, y i * u ^ i| ≤ K * u ^ R)
  (hL : u Set.Icc (0 : ) b, |u - x 0| ≤ L * u) :
  C : , 0 ≤ C u Set.Icc (0 : ) b, s, 0 ≤ s →
  | u * Real.exp ( s * u)
    - ∑ j range R, u ^ j * (Real.exp ( s * x 0) * ampJet x y R j s)|
  ≤ C * u ^ R * (1 + s) ^ R * Real.exp ( s * (x 0 + L * b))
```

2 Mathematics

Write $a = x_0$, $g = \xi - a$, $E = e^{\beta s g(u)}$, $E_{\text{jet}} = \sum_{n < R} E_n(s) u^n$ and $\eta_P = \sum_{i < R} y_i u^i$. Then

$$\eta e^{\beta s \xi} - \sum_{j < R} u^j e^{\beta s a} P_j(s) = e^{\beta s a} \left[(\eta - \eta_P) E + \eta_P (E - E_{\text{jet}}) + (\eta_P E_{\text{jet}} - \sum_{j < R} P_j(s) u^j) \right].$$

The first piece is bounded by $K u^R e^{\beta L_1 b s}$ (Taylor remainder of η times the exponential envelope of E). The second is $|\eta_P|$ times the exponential jet remainder of the previous unit, with $|\eta_P| \leq \sum |y_i| b^i$. The third is the truncation error of the polynomial $\text{jetPoly}(y) \cdot \text{jetPoly}(E(s))$, whose coefficients below R are precisely the Cauchy products $P_j(s)$ (`coeff_jetPoly_mul`, via `Polynomial.coeff_mul` and `Finset.Nat.sum_antidiagonal_eq_sum_range_succ`). The product has degree $\leq 2R - 2$, so `poly_trunc_le` sums over $n \in [R, 2R)$; each coefficient is bounded by $(n + 1)(\sum |y_i|)(\sum_l |E_l(s)|)$, and $\sum_l |E_l(s)| \leq (\sum_l \exp \text{JetConst}_l)(1 + s)^R$ by the previous unit's envelope. Collecting, $C = K + Y_b C_1 + T Y E_C$ with $T = \sum_{n \in [R, 2R)} (n + 1) b^{n-R}$.

The hypothesis $0 < R$ is needed only to identify $\text{eval}(g_R) = \sum_{i < R} x_i u^i - x_0$ (`eval_gPoly_eq`); for $R = 0$ the statement is vacuous anyway.

3 Paper-facing meaning

With unit 92 (`oneDKernel_jet_expansion`: an abstract all-order transfer from a u -jet with remainder $u^R H_R(s)$ to an expansion of $\int_0^b u^h e^{-\beta(Nu^k)^2} F(u, Nu^k) du$ in $N^{-(p+j/k)}$) this unit supplies exactly the needed input for the chart amplitude $F(u, s) = \eta(u) e^{\beta s \xi(u)}$: $F_j(s) = e^{\beta s x_0} P_j(s)$ and $H_R(s) = C(1 + s)^R e^{\beta s(x_0 + L_1 b)}$. The remaining step for the all-order 1D theorem (unit 96) is the moment integrability of $s^\gamma e^{-\beta s^2} \cdot s^\ell e^{\beta s a'}$, which is the polynomial-bound generalisation of `moment_integrableOn_of_bound'`, plus the identification of the coefficients $c_j = k^{-1} \sum_\ell p_{j,\ell} S_{qj+\ell}(x_0)$ in the $S_\lambda(a)$ notation of the paper.

4 Lean notes

- `rw [Finset.sum_mul]` matched an inner product $(\sum |y_i|) \cdot \sum |E_l|$ instead of the intended outer one; pass the `Finset` explicitly, `Finset.sum_mul (Ico R (2 * R))`.
- `rw [Finset.mul_sum]` unfolds `set-bound` variables whose value is a sum (zeta-delta reduction during matching), distributing $\eta_P \cdot E_{\text{jet}}$ unexpectedly. Prove the sum identity as a separate `have` on the raw expression and rewrite with it.
- `Finset.sum_congr rfl fun n _ =>` by ring inside a `calc` step left the `AddCommMonoid` instance stuck; refine `Finset.sum_congr rfl ?_ ; intro n _ ; ring` works.
- `Polynomial.natDegree_sum_le_of_forall_le` and `natDegree_C_mul_X_pow_le` give the degree bound $\leq R - 1$ in two lines; combined with `natDegree_mul_le` the product's coefficient support is inside $[0, 2R)$ by `omega`.